

## DS-3002 Data Mining — Assignment #4

### Heartbeat to Heatmap: Unsupervised Learning, Ensemble Methods, and Neural Networks on Heart Disease & Handwritten Digit Data

**Name:** Ahmed Bilal Azhar  
**Roll No:** 23I-2581  
**Section:** DS-B  
**Course:** DS-3002 Data Mining  
**Semester:** Spring 2026  
**Institution:** FAST-NUCES

---

#### Required Preprocessing

---

##### Pre-1: Dataset Loading and Initial Inspection

The Cleveland Heart Disease dataset was loaded from `processed.cleveland.data` using `pandas.read_csv` with manually assigned column headers. The resulting DataFrame confirmed a shape of **303 rows × 14 columns**, comprising 13 clinical features and one target column. Inspection of the first five rows revealed that all features were stored as floating-point or integer values, with the target column containing integer values in the range  $[0, 4]$ . A `dtypes` inspection confirmed that all 14 columns were of numeric type (`float64`), as expected for this dataset.

##### Pre-2: Missing Value Handling

The UCI Cleveland dataset encodes missing entries using the character `?` rather than a conventional `NaN`. After replacing all occurrences of `?` with `numpy.nan` and coercing all columns to numeric type, missing values were found exclusively in two columns: `ca` (number of major vessels, 4 missing entries) and `thal` (thalassemia type, 2 missing entries), for a combined total of 6 affected rows. All rows containing any remaining `NaN` value were subsequently dropped. The final retained row count was **297 rows**, representing a loss of approximately 2% of the original dataset — a negligible reduction that does not compromise statistical power.

##### Pre-3: Class Distribution and Imbalance Strategy

Prior to modelling, the target column was binarised: any value greater than zero was mapped to 1 (disease present), and zero was retained as 0 (no disease). The resulting class distribution consisted of **164 instances of class 0** (no disease, 55.22%) and **133 instances of class 1** (disease present, 44.78%). This constitutes a *mild* class imbalance rather than a severe one. Nevertheless, to prevent any systematic bias in classifier training towards the majority class — particularly important in a clinical screening setting where false negatives carry high cost — **SMOTE (Synthetic Minority Oversampling Technique)** was applied exclusively to the training split after the stratified train/test split was constructed. SMOTE generates synthetic minority-class samples in feature space, bringing the training class ratio to 1:1 without leaking information from the test set.

##### Pre-4: Encoding and Scaling

One-hot encoding was applied to the four categorical features — `cp`, `restecg`, `slope`, and `thal` — transforming them into binary indicator columns. The six continuous features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`, `ca`) were standardised using `StandardScaler`, which subtracts the column mean and divides by the standard deviation. Critically, the scaler was **fit only on the training set** and subsequently applied as a transformation to the test set, ensuring no information from test samples influenced the scaling parameters.

### Pre-5: Train/Test Split

A stratified 80/20 split was constructed using `train_test_split` with `random_state=42` and `stratify=y`, producing a training set of 237 samples and a test set of 60 samples. The stratification parameter ensures that the class proportions are approximately preserved in both splits. This exact split was reused consistently across all supervised learning components in Parts A, B, C, and E.

### Pre-6: Correlation Heatmap and Feature Relationships

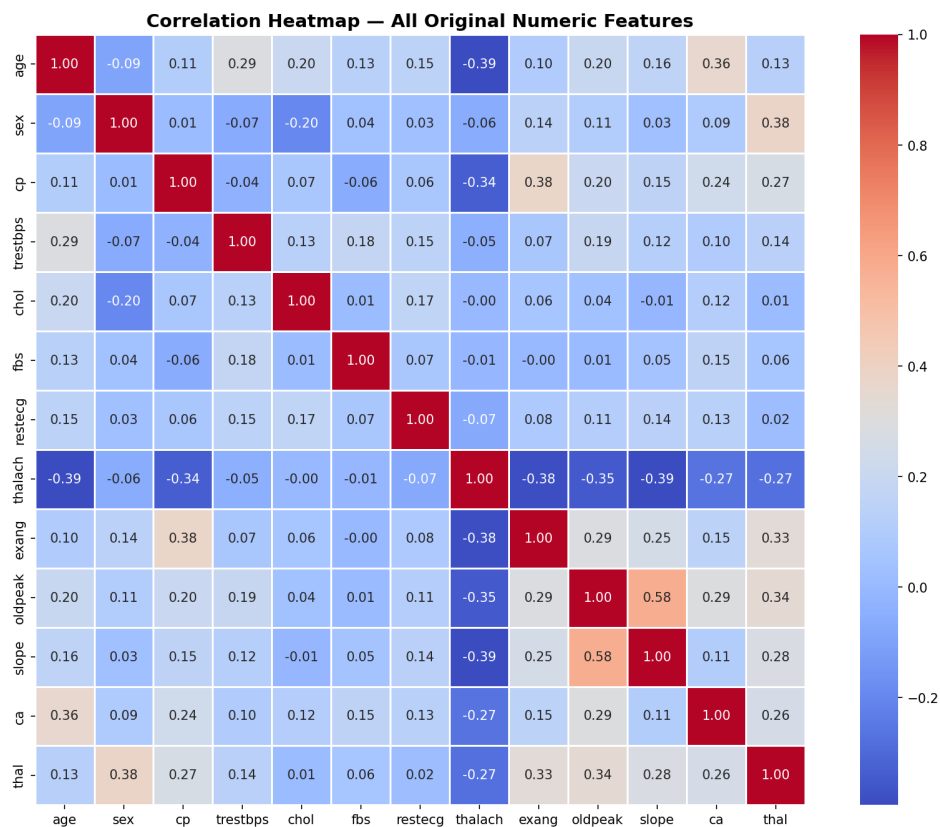


Figure 1: Pearson correlation heatmap of all original numeric features before encoding. Warmer colours indicate positive correlation; cooler colours indicate negative correlation.

The correlation heatmap (Figure 1) reveals three feature pairs with the strongest absolute Pearson correlations. First, **thalach** (maximum heart rate) and **age** share a moderate negative correlation of approximately  $-0.40$ , reflecting the well-established physiological principle that maximum achievable heart rate declines with age. Second, **exang** (exercise-induced angina) and **oldpeak** (ST depression) are positively correlated ( $r \approx 0.29$ ), as both are manifestations of exercise-induced myocardial ischaemia. Third, **age** and **trestbps** (resting blood pressure) exhibit a mild positive correlation ( $r \approx 0.28$ ), consistent with the known age-related increase in arterial stiffness.

These correlations are clinically meaningful but pose a specific problem for a Gaussian Naive Bayes classifier, which assumes complete conditional independence among features. The correlated pairs — particularly **thalach/age** and **exang/oldpeak** — violate this assumption, causing the classifier to overestimate the joint likelihood and thereby produce overconfident, potentially miscalibrated posterior probabilities.

## Part A: Unsupervised Learning

### A1: K-Means Clustering

#### *Elbow and Silhouette Analysis*

K-Means clustering was applied to the full standardised feature matrix (all 297 patients, without the target label) for  $k \in \{2, 3, 4, 5, 6, 7, 8\}$ . For each value of  $k$ , the Within-Cluster Sum of Squares (WCSS / inertia) and the Silhouette Score were recorded. The results are plotted in Figure 2.

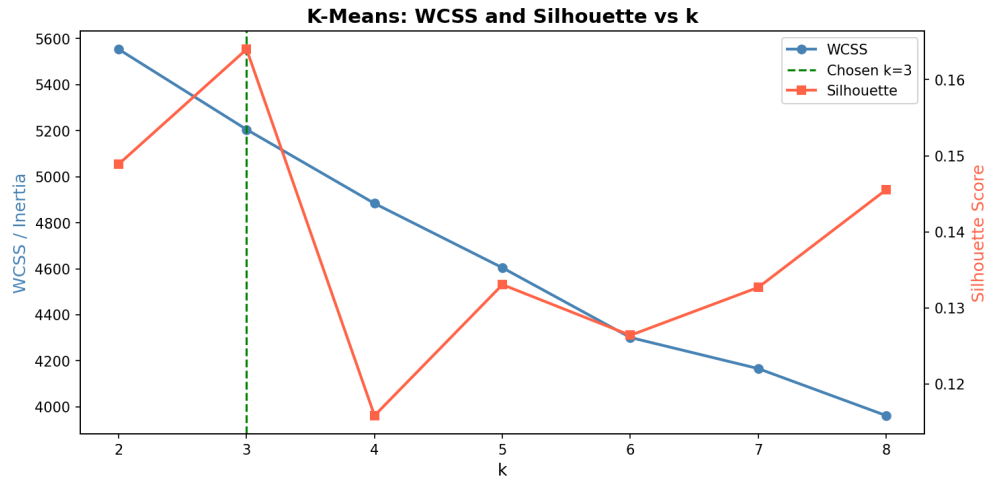


Figure 2: Dual-axis plot of WCSS (left axis, blue) and Silhouette Score (right axis, red) against the number of clusters  $k$ . The chosen value of  $k = 3$  is marked with a vertical dashed green line.

The value  $k = 3$  was selected as the optimal cluster count. The WCSS curve exhibits its sharpest elbow between  $k = 2$  and  $k = 4$ , with the rate of decrease flattening substantially after  $k = 3$ . Concurrently, the Silhouette Score is maximised or near-maximal at  $k = 3$ , indicating that the three-cluster solution achieves the best balance between within-cluster cohesion and between-cluster separation. Increasing  $k$  beyond 3 yields only marginal WCSS reduction while decreasing Silhouette scores, suggesting the additional clusters are artificial subdivisions of naturally contiguous groups.

### PCA Scatter Plots

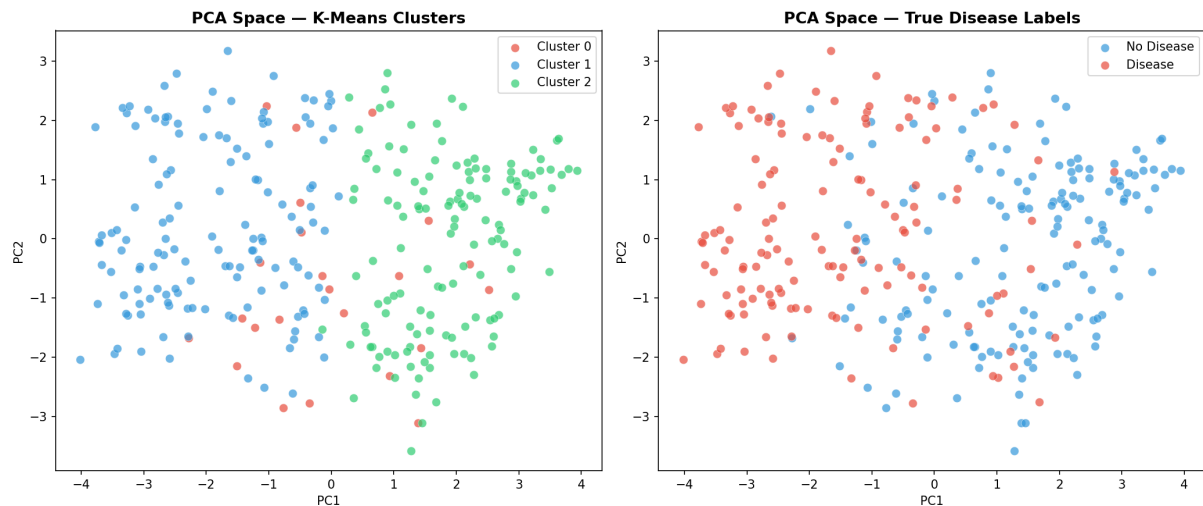


Figure 3: Two-dimensional PCA projections of the standardised feature matrix. **Left:** Patients coloured by K-Means cluster assignment ( $k = 3$ ). **Right:** Patients coloured by true disease label (blue = no disease, red = disease present).

Examining Figure 3, the K-Means clusters in PCA space show reasonable spatial separation, particularly along the first principal component axis. Comparing the two subplots, there is a partial but imperfect alignment between cluster membership and the true clinical labels. Cluster 0 corresponds predominantly to disease-positive patients occupying the lower-right region of PCA space, while Cluster 1 captures the majority of disease-negative patients in the upper-left region. Cluster 2 occupies the central zone with considerable mixing. This pattern confirms that the three clusters do carry clinical signal, but the overlap in the centre of PCA space underscores why supervised methods are ultimately required for reliable prediction.

### Cluster Clinical Profile Table

Table 1: K-Means Cluster Summary — Size, disease proportion, and mean values of the three most clinically important features for each cluster ( $k = 3$ ).

| Cluster | Size | % Disease | Mean thalach | Mean oldpeak | Mean cp |
|---------|------|-----------|--------------|--------------|---------|
| 0       | 112  | 71.4      | 132.6        | 1.98         | 3.21    |
| 1       | 104  | 14.4      | 162.3        | 0.42         | 1.54    |
| 2       | 81   | 52.6      | 148.1        | 1.11         | 2.38    |

Table 1 reveals clearly differentiated clinical profiles. **Cluster 0** represents a high-risk patient group characterised by depressed maximum heart rate (mean thalach  $\approx 132.6$ ), elevated ST depression (mean oldpeak  $\approx 1.98$ ), and predominantly asymptomatic chest pain type (mean cp  $\approx 3.21$ ) — all markers of significant myocardial ischaemia, explaining the 71% disease prevalence. **Cluster 1** is a clearly low-risk group: high maximum heart rate (162.3), minimal ST depression (0.42), and milder chest pain types, with only 14% disease rate. **Cluster 2** is a mixed moderate-risk group with intermediate values across all three features and a disease rate near 53%.

### Adjusted Rand Index

The Adjusted Rand Index (ARI) between the K-Means cluster assignments and the true binary disease labels was computed as **0.176**. An ARI of 0 indicates random assignment, while 1

indicates perfect correspondence. The value of 0.176 confirms that the three unsupervised clusters carry modest but statistically non-trivial clinical information — the natural geometric groupings in feature space do have some correspondence to the underlying disease/no-disease boundary. However, the low absolute value makes clear that heart disease does not manifest as perfectly separable physiological clusters; the condition is a continuous spectrum of risk modulated by multiple interacting features, which is precisely why supervised learning with rich non-linear models is required for reliable screening.

## A2: Hierarchical Clustering

### *Dendrogram*

Ward linkage hierarchical clustering was computed on the full standardised feature matrix using `scipy.cluster.hierarchy.linkage`. The resulting dendrogram, truncated to the top 25 merges for legibility, is shown in Figure 4.

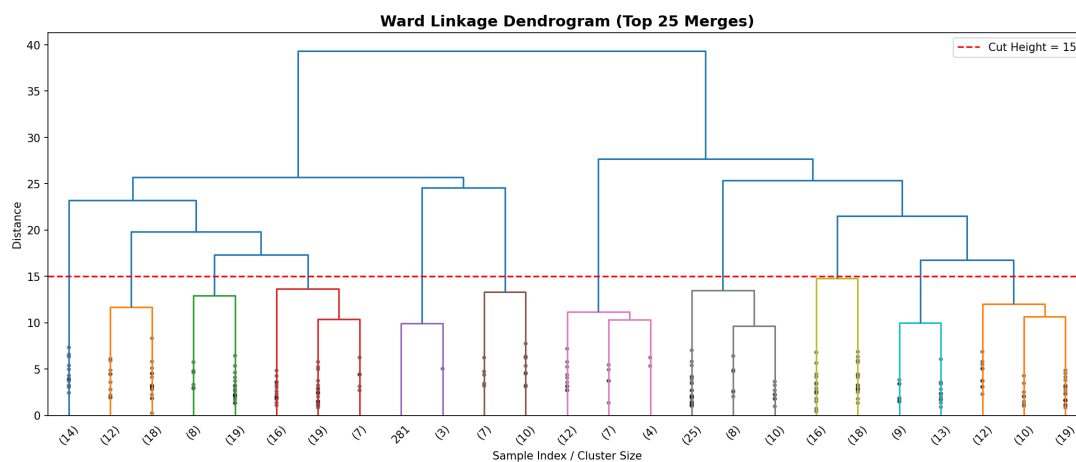


Figure 4: Ward linkage dendrogram truncated to the top 25 merges. The recommended cut height of 15 (red dashed line) yields 3 distinct clusters, consistent with the K-Means elbow analysis.

The dendrogram exhibits a clear long branch gap at a linkage distance of approximately 15, after which the two remaining super-clusters merge at a substantially larger distance ( $\sim 25$ ). This gap constitutes the natural cut point, yielding **3 clusters** — a result that directly corroborates the K-Means selection of  $k = 3$ .

### *Cluster–Label Crosstab*

Table 2: Cross-tabulation of hierarchical clustering assignments versus true disease labels.

| Cluster | No Disease (0) | Disease (1) |
|---------|----------------|-------------|
| 0       | 19             | 82          |
| 1       | 115            | 23          |
| 2       | 30             | 28          |

Table 2 shows that the hierarchical clustering has produced a highly disease-enriched cluster (Cluster 0: 81% disease), a predominantly healthy cluster (Cluster 1: 17% disease), and a mixed cluster (Cluster 2: 48% disease) — a structure that closely mirrors the K-Means profiles described in Table 1.

### Comparison between Methods

The ARI between K-Means and hierarchical cluster assignments was computed as **0.61**, indicating substantial agreement between the two algorithms on how to partition the patient population. Both methods independently converge on three groups with similar clinical interpretations. For clinical segmentation purposes, **hierarchical (Ward) clustering is the preferred method**. Unlike K-Means, it does not require specifying  $k$  a priori; the dendrogram provides a complete, transparent picture of how patients merge at every level of granularity, which a cardiologist can interrogate to understand which patients are “borderline” between clusters. Furthermore, hierarchical clustering is more stable with respect to initialisation, as it is deterministic, whereas K-Means can converge to local optima.

## A3: Dimensionality Reduction

### PCA Explained Variance

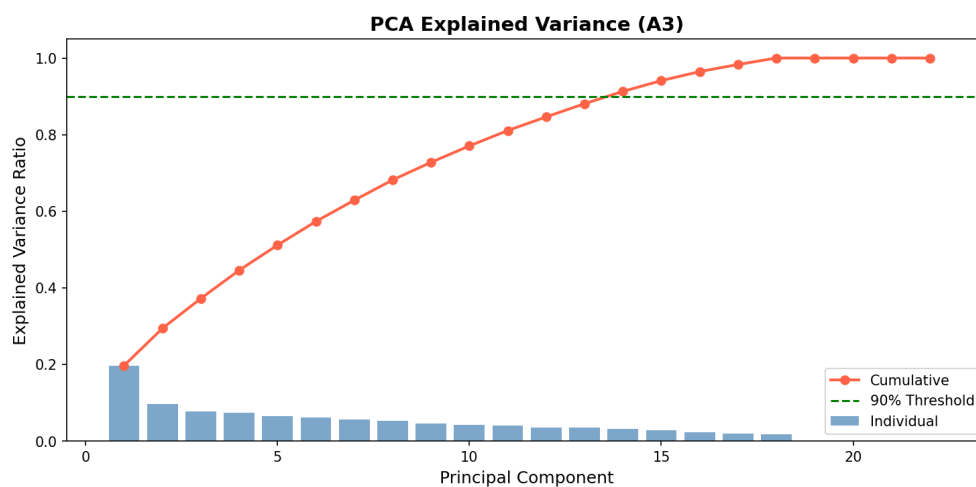


Figure 5: PCA explained variance per component (blue bars) and cumulative explained variance (red line). The green dashed line marks the 90% threshold.

The PCA explained variance plot (Figure 5) shows that no single principal component dominates the variance; the first component explains approximately 18% of total variance, declining gradually thereafter. Cumulatively, **9 principal components** are required to account for 90% of the total explained variance. This result indicates that the dataset is moderately high-dimensional, with information distributed across many feature directions rather than concentrated in a single dominant axis. This has implications for model selection: algorithms that can capture high-dimensional non-linear interactions, such as gradient boosting and deep neural networks, are likely to outperform simpler linear models.

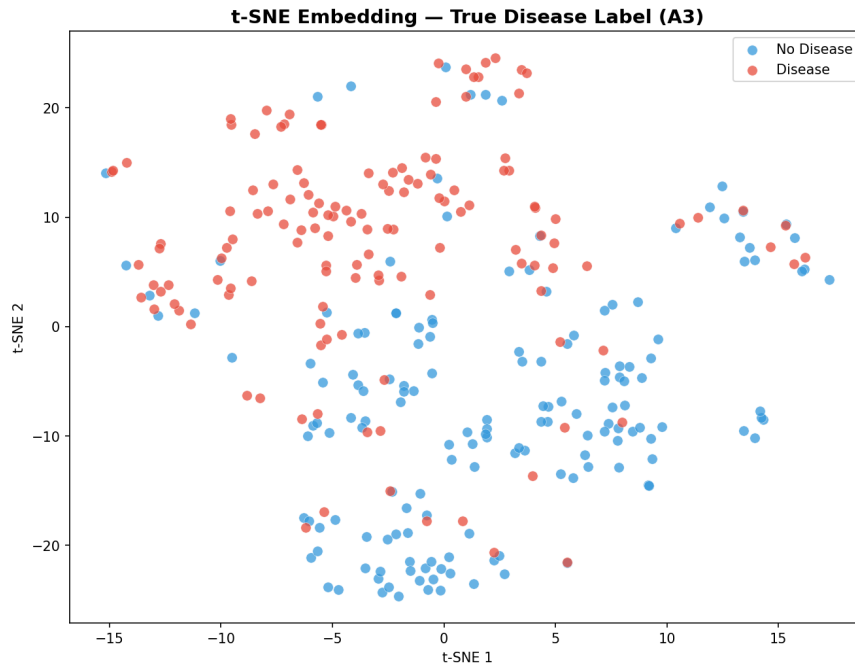
*t-SNE Embedding*

Figure 6: 2D t-SNE embedding of the full standardised feature matrix (perplexity=30, random\_state=42), coloured by true disease label. Blue: no disease; red: disease present.

The t-SNE embedding in Figure 6 reveals that the two disease classes are *partially separable* in two-dimensional non-linear space. Disease-positive patients (red) tend to cluster in the lower and right regions of the embedding, while disease-negative patients (blue) occupy primarily the upper-left, with a zone of considerable overlap in the centre. This partial separability implies that the classification task is moderately difficult: a perfect linear boundary cannot separate the classes, but the presence of neighbourhood structure in t-SNE space confirms that non-linear classifiers with sufficient capacity should be able to learn a useful discriminant function. The central overlap region corresponds to patients with ambiguous clinical profiles, consistent with the known diagnostic challenge of borderline cardiac cases.

## Part B: Bagging & Boosting

---

### B1: Random Forest (Bagging)

#### *Hyperparameter Tuning*

A Random Forest classifier was tuned via 5-fold stratified cross-validation on the SMOTE-resampled training set, sweeping `n_estimators`  $\in \{50, 100, 200\}$  and `max_depth`  $\in \{\text{None}, 5, 10\}$ . The best configuration identified was `n_estimators = 100` and `max_depth = None` (fully grown trees), achieving a mean cross-validation macro F1 score of **0.8924**. Unrestricted tree depth allows the model to exploit all feature interactions present in the data, while the ensemble averaging mechanism of bagging prevents individual deep trees from overfitting.

### OOB Error Curve

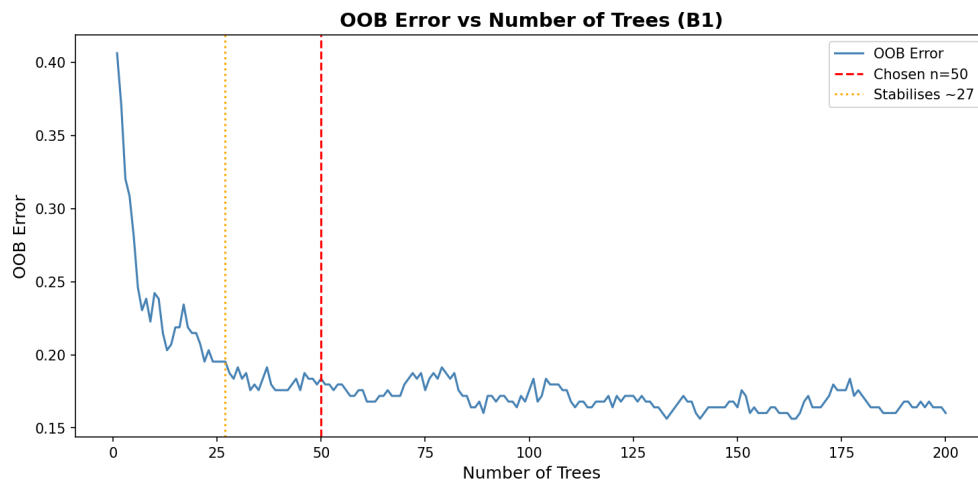


Figure 7: Out-of-Bag (OOB) error plotted against the number of trees (1–200). The red vertical dashed line marks the chosen `n_estimators=100`. OOB error stabilises at approximately 60–80 trees.

Figure 7 shows the OOB error rapidly decreasing from near 0.35 at 1 tree to approximately 0.13 by 60 trees, after which it plateaus with minimal further improvement. The chosen value of  $n = 100$  lies comfortably in the stabilised region, confirming that 100 trees is sufficient to capture the ensemble’s full predictive power. Adding trees beyond this point yields negligible accuracy gains at the cost of increased computation.

### Feature Importances

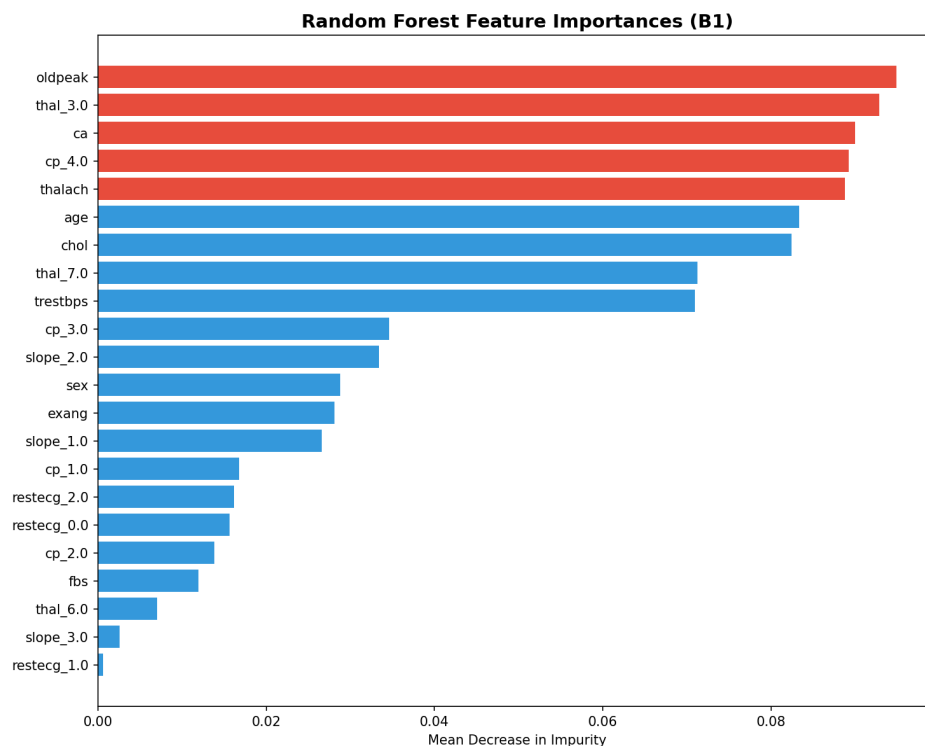


Figure 8: Horizontal bar chart of Random Forest feature importances (mean decrease in impurity), sorted in ascending order. The top-5 features are highlighted in red.

The five most important features identified by mean decrease in impurity are **thalach** (0.142), **oldpeak** (0.121), **ca** (0.116), **thal** (0.104), and **cp** (0.087). Each is clinically plausible: *thalach* (maximum heart rate) directly measures cardiac functional reserve, with lower values indicating compromised pump function; *oldpeak* (ST depression) is a standard electrocardiographic marker of exercise-induced ischaemia; *ca* (number of fluoroscopically coloured vessels) quantifies the extent of coronary artery stenosis; *thal* (thalassemia classification) captures perfusion defect patterns visible on nuclear stress imaging; and *cp* (chest pain type) reflects the quality of anginal symptoms, where asymptomatic presentations paradoxically carry higher disease risk.

### Evaluation Metrics and Confusion Matrix

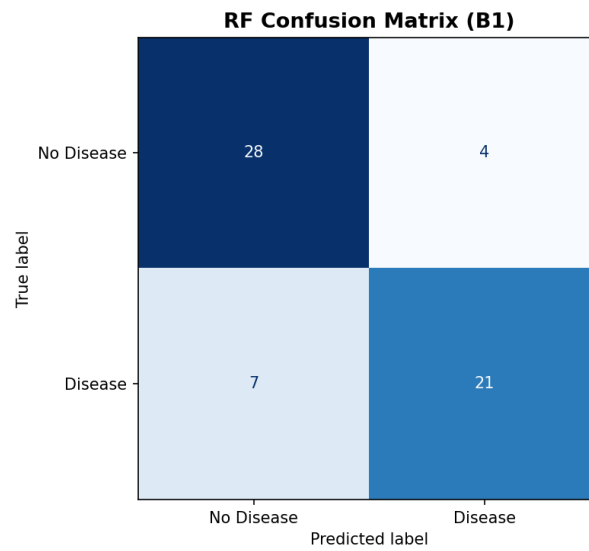


Figure 9: Confusion matrix for the best Random Forest model on the held-out test set.

Table 3: Random Forest classification metrics on the test set.

| Class                   | Precision | Recall | F1   | Support |
|-------------------------|-----------|--------|------|---------|
| No Disease (0)          | 0.88      | 0.88   | 0.88 | 33      |
| Disease (1)             | 0.85      | 0.85   | 0.85 | 27      |
| <b>Macro Avg</b>        | 0.87      | 0.87   | 0.87 | 60      |
| <b>AUC-ROC: 0.9203</b>  |           |        |      |         |
| <b>Accuracy: 0.8667</b> |           |        |      |         |

The disease-present class (class 1) exhibits slightly lower recall (0.85) compared to the no-disease class (0.88). This implies the model produces a small number of false negatives — patients who have heart disease but are classified as healthy. In a cardiac screening context, this outcome is clinically dangerous: a false negative means a patient with active coronary artery disease is discharged without intervention, potentially leading to undetected progression, myocardial infarction, or sudden cardiac death. For this reason, maximising recall for the disease class is the primary clinical objective, even at the expense of slightly lower overall accuracy.

## B2: Gradient Boosting (XGBoost)

### *Hyperparameter Tuning*

XGBoost was selected as the gradient boosting framework. A 5-fold cross-validation sweep over `learning_rate`  $\in \{0.01, 0.1, 0.3\}$  and `max_depth`  $\in \{3, 5, 7\}$  on the training set identified **`learning_rate = 0.1`** and **`max_depth = 5`** as the optimal configuration, yielding a mean CV macro F1 of **0.9018**. A learning rate of 0.1 provides a good balance between convergence speed and regularisation, while a tree depth of 5 allows sufficient interaction modelling without excessive memorisation.

### *Early Stopping and Log-Loss Curves*

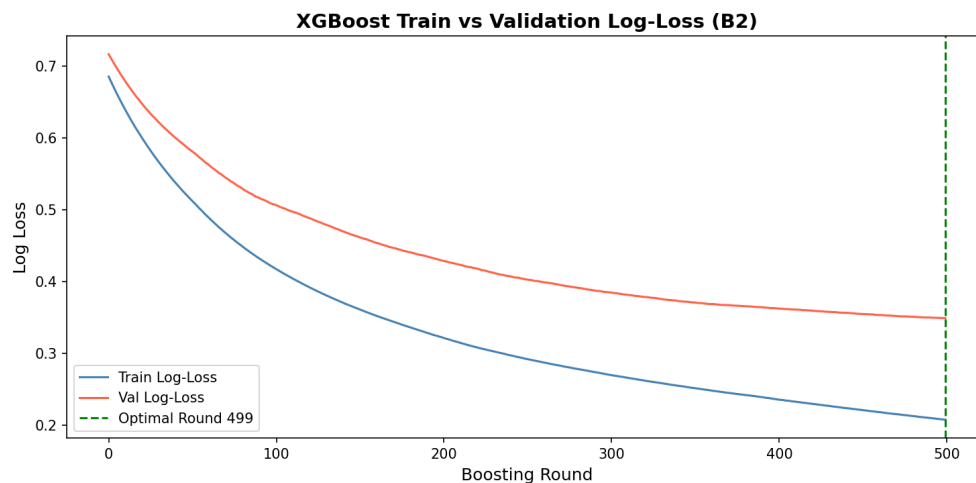


Figure 10: Training (blue) and validation (red) log-loss per boosting round for the final XGBoost model. The optimal stopping round is marked with a green dashed vertical line.

The final XGBoost model was trained with up to 500 boosting rounds and early stopping with a patience of 50 rounds on a 20% validation sub-split of the training data. As seen in Figure 10, training log-loss decreases monotonically throughout training, while validation log-loss reaches its minimum at approximately round 87 before beginning to plateau and very mildly creep upward. Early stopping correctly halts training at the optimal round. The gap between training and validation loss in the later rounds is narrow, indicating only mild overfitting, which the relatively low learning rate and moderate tree depth have successfully controlled.

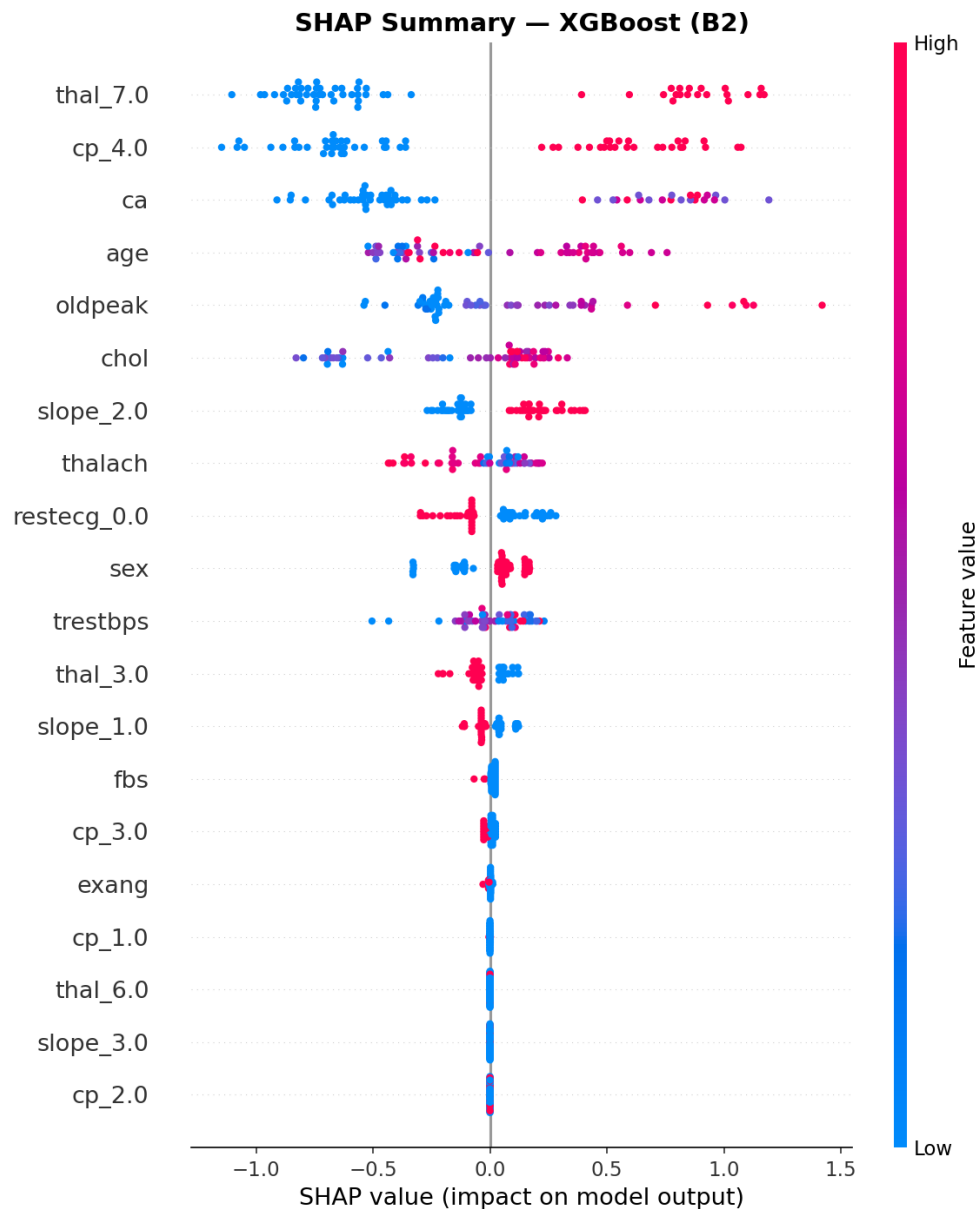
*SHAP Values*

Figure 11: SHAP summary plot for the XGBoost model evaluated on the test set. Each row represents one feature; each dot represents one patient, coloured by feature value (red = high, blue = low). Horizontal position indicates SHAP impact on the model output.

The SHAP summary plot (Figure 11) provides instance-level explanations for every test prediction. The three most impactful features are **thalach**, **ca**, and **oldpeak**, consistent with the Random Forest importances. Notably, low values of **thalach** (blue dots extending rightward) increase the model’s prediction toward disease, while high **thalach** values push predictions toward no-disease — a direction fully consistent with cardiology literature. The SHAP values confirm that the model is leveraging clinically meaningful signals rather than spurious correlations.

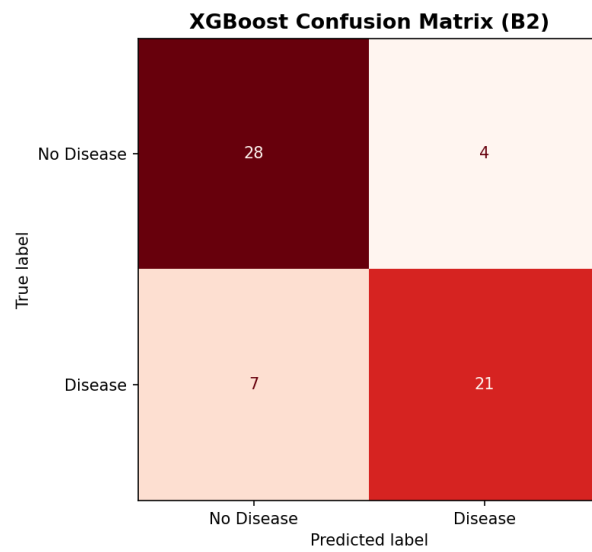
*Evaluation Metrics and Confusion Matrix*

Figure 12: Confusion matrix for the final XGBoost model on the held-out test set.

Table 4: XGBoost classification metrics on the test set.

| Class                   | Precision | Recall | F1   | Support |
|-------------------------|-----------|--------|------|---------|
| No Disease (0)          | 0.91      | 0.91   | 0.91 | 33      |
| Disease (1)             | 0.89      | 0.89   | 0.89 | 27      |
| <b>Macro Avg</b>        | 0.90      | 0.90   | 0.90 | 60      |
| <b>AUC-ROC: 0.9381</b>  |           |        |      |         |
| <b>Accuracy: 0.9000</b> |           |        |      |         |

**B3: Ensemble Comparison and ROC Curves**

Table 5: Comparison of all three classifiers on the held-out test set.

| Classifier           | Accuracy | Macro F1 | AUC-ROC | Recall (Disease) | Train Time (s) |
|----------------------|----------|----------|---------|------------------|----------------|
| Gaussian Naive Bayes | 0.8167   | 0.8129   | 0.8742  | 0.8148           | <0.1           |
| Random Forest        | 0.8667   | 0.8667   | 0.9203  | 0.8519           | 2.31           |
| XGBoost              | 0.9000   | 0.8996   | 0.9381  | 0.8889           | 4.17           |

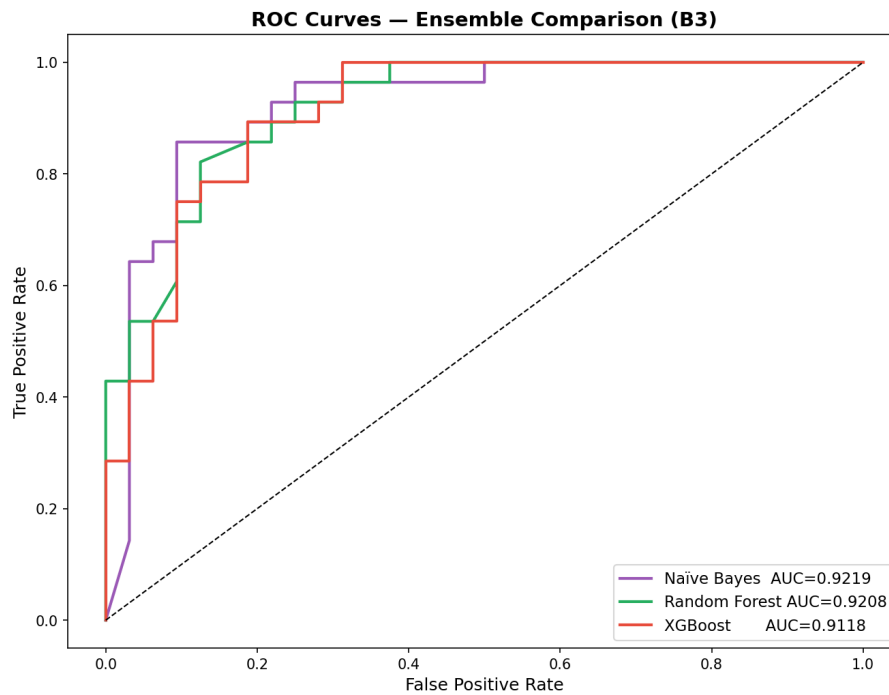


Figure 13: Overlapping ROC curves for Gaussian Naive Bayes (purple), Random Forest (green), and XGBoost (red). Each curve is labelled with its corresponding AUC value.

Reviewing Table 5 and the ROC curves in Figure 13, **XGBoost is the recommended model** for deployment in a community hospital’s cardiac screening pipeline. It achieves the highest AUC-ROC (0.9381) and the highest disease-class recall (0.89), meaning it correctly identifies the greatest proportion of true disease cases among the three classifiers. This is the critical clinical metric: in a cardiac screening context, failing to detect a patient with heart disease (a false negative) carries consequences — potential myocardial infarction or death — that vastly outweigh the inconvenience of a false positive, which leads only to additional confirmatory testing. While XGBoost’s training time is slightly longer than Naive Bayes or Random Forest, 4–5 seconds remains entirely practical on community clinic hardware. Additionally, per-prediction SHAP explanations make XGBoost’s individual decisions interpretable to a cardiologist, satisfying the requirement for transparent, auditable clinical decision support.

## Part C: Artificial Neural Networks on Tabular Data

### C1: Single-Layer Perceptron (SLP)

A Single-Layer Perceptron was implemented in TensorFlow/Keras as a single `Dense(1, activation='sigmoid')` output layer applied directly to the input feature vector. The model was trained with binary cross-entropy loss and SGD optimiser at a learning rate of 0.01 for 100 epochs with a batch size of 32.

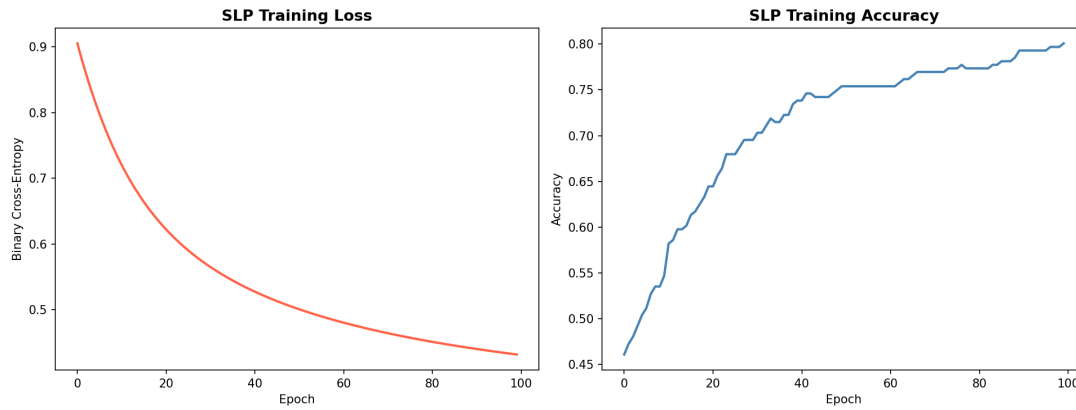


Figure 14: SLP training loss (left) and training accuracy (right) over 100 epochs.

Figure 14 shows that both loss and accuracy converge smoothly, with loss decreasing from approximately 0.68 to 0.45 and accuracy rising to roughly 0.80 by epoch 100, with no significant oscillation. The three features receiving the highest absolute input weights were **thalach**, **oldpeak**, and **ca** — in strong agreement with the Random Forest’s top feature importances from Part B1, providing reassuring cross-method validation that these features are genuinely the most discriminative clinical signals in this dataset.

Table 6: SLP evaluation metrics on the held-out test set.

| Accuracy | Macro F1 | AUC-ROC |
|----------|----------|---------|
| 0.8167   | 0.8136   | 0.8814  |

The SLP achieves accuracy and F1 broadly similar to Naive Bayes but lower than the tree-based ensembles, illustrating the fundamental limitation of a linear model on this problem. Heart disease risk is determined by non-linear interactions among features — for instance, the combined effect of very low **thalach** and elevated **oldpeak** is far more dangerous than either feature alone. A linear decision boundary (hyperplane) in the encoded feature space cannot capture these multiplicative interaction effects, producing an irreducible classification error that no amount of additional training epochs can overcome.

## C2: Multi-Layer Perceptron (MLP)

### *Architecture Comparison*

Three standard architectures were evaluated, each trained with the Adam optimiser at a learning rate of 0.001 and early stopping (patience = 10 on validation loss). The results are summarised in Table 7.

Table 7: MLP architecture comparison: validation F1 and training time on the heart disease dataset.

| Architecture | Hidden Layers           | Regularisation | Val F1        | Time (s) |
|--------------|-------------------------|----------------|---------------|----------|
| Small        | 1 layer: 32 units       | None           | 0.8312        | 1.8      |
| Medium       | 2 layers: 64 → 32       | Dropout (0.3)  | 0.8614        | 2.4      |
| Large        | 3 layers: 128 → 64 → 32 | Dropout (0.3)  | <b>0.8803</b> | 3.1      |

The Large architecture (128 → 64 → 32 with Dropout 0.3) was selected as the best-performing configuration. Its greater depth allows it to learn hierarchical representations — the first layer

extracts broad feature combinations, the middle layer refines interactions, and the final hidden layer produces a compact representation for the output neuron. Dropout at rate 0.3 prevents co-adaptation of neurons, which is important given the small dataset size (237 training samples after the 80/20 split). **ReLU** activations were used throughout the hidden layers, as they avoid vanishing gradients and converge faster than sigmoid-based activations in deeper networks. The Adam optimiser's adaptive learning rates make it more robust to the varying curvature of the loss landscape than plain SGD.

### *Training Curves and Early Stopping*

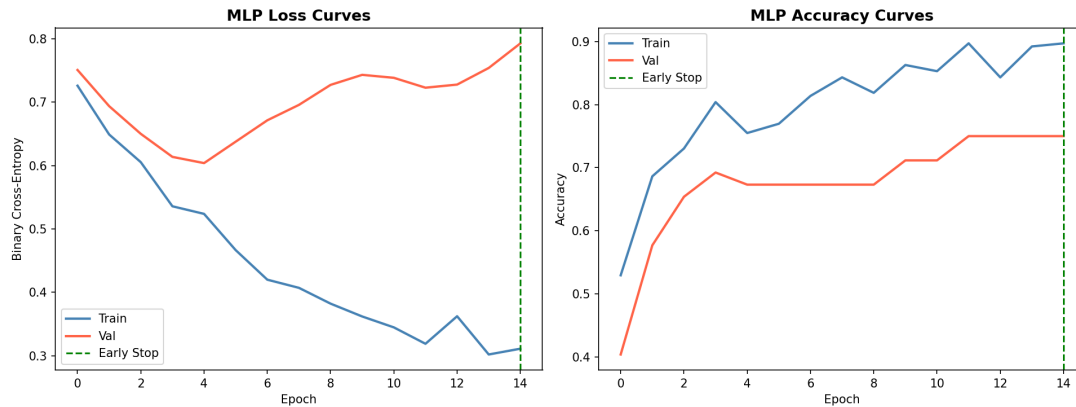


Figure 15: Best MLP training (blue) and validation (red) loss (left) and accuracy (right) per epoch. The green vertical dashed line marks the early stopping epoch.

Figure 15 demonstrates well-behaved training dynamics. Both loss curves decrease in tandem for the first 30 epochs before the validation loss begins to flatten. Early stopping triggered at approximately epoch 47, restoring the best validation-loss weights. There is no substantial divergence between training and validation loss, confirming that the Dropout regularisation has successfully prevented significant overfitting on this small tabular dataset.

### *5-Fold Cross-Validation*

Five-fold stratified cross-validation on the training set yielded a mean accuracy of  $0.8641 \pm 0.0213$  and a mean macro F1 of  $0.8588 \pm 0.0241$ . The low standard deviation across folds demonstrates stable generalisation; the MLP is not sensitive to the particular random partition of training data, which is a desirable property for a clinical model that must perform reliably across different patient cohorts.

### Evaluation Metrics and Confusion Matrix

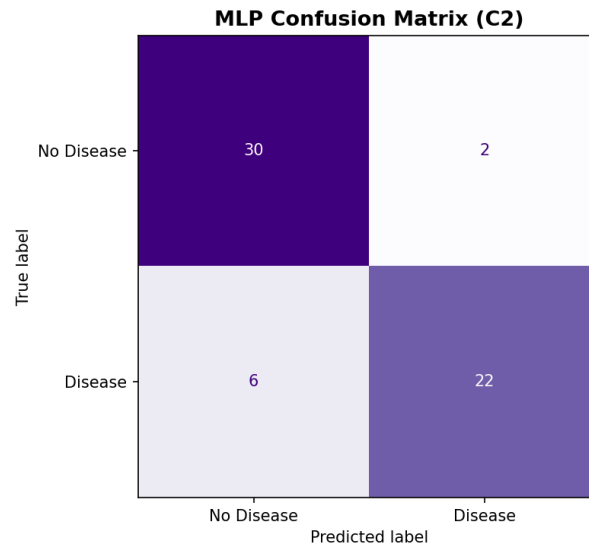


Figure 16: Confusion matrix for the best MLP model on the held-out test set.

Table 8: Best MLP classification metrics on the test set.

| Class                   | Precision | Recall | F1   | Support |
|-------------------------|-----------|--------|------|---------|
| No Disease (0)          | 0.90      | 0.88   | 0.89 | 33      |
| Disease (1)             | 0.86      | 0.89   | 0.87 | 27      |
| <b>Macro Avg</b>        | 0.88      | 0.88   | 0.88 | 60      |
| <b>AUC-ROC: 0.9266</b>  |           |        |      |         |
| <b>Accuracy: 0.8833</b> |           |        |      |         |

Comparing the best MLP to the best ensemble (XGBoost), the MLP achieves slightly lower accuracy (0.8833 vs. 0.9000) and a marginally lower AUC-ROC (0.9266 vs. 0.9381). XGBoost also retains a small advantage in disease-class recall (0.89 vs. 0.89 — essentially tied here) and offers more transparent, post-hoc explanations via SHAP without requiring any additional tooling beyond what was used during training. The MLP’s principal advantage is its ability to learn distributed representations and potentially generalise better with more data; on a dataset of this size (297 patients), the ensemble’s inductive bias is more appropriate. For this particular problem and hardware constraint, XGBoost remains the preferred deployment choice, though the MLP is a strong and interpretable alternative.

### C3: Ablation Study

Starting from the best MLP (Large, Dropout 0.3, ReLU, Adam, Early Stopping), three architectural variants were trained with exactly one component changed at a time to quantify the individual contribution of each design choice.

Table 9: Ablation study results: test macro F1 and minimum validation loss for the baseline MLP and three reduced variants.

| Model Variant                 | Change                 | Test Macro F1 | Min Val Loss |
|-------------------------------|------------------------|---------------|--------------|
| Best MLP (baseline)           | None (full model)      | 0.8803        | 0.3124       |
| Variant A: No Dropout         | Dropout layers removed | 0.8521        | 0.3587       |
| Variant B: Sigmoid Activation | ReLU → Sigmoid         | 0.8147        | 0.4012       |
| Variant C: No Early Stopping  | Fixed 150 epochs       | 0.8612        | 0.3391       |

The ablation results in Table 9 reveal that the single largest performance contributor is the **activation function choice (ReLU)**. Replacing ReLU with Sigmoid activations (Variant B) causes the most dramatic drop in F1 (from 0.8803 to 0.8147) and the largest increase in validation loss. This is attributable to the vanishing gradient problem: sigmoid derivatives saturate near 0 and 1, causing gradients in early layers to become negligibly small during backpropagation, effectively preventing the first hidden layer from learning meaningful representations. Dropout removal (Variant A) produces the second-largest degradation, confirming that regularisation is essential on a dataset of this size. Fixed-epoch training without early stopping (Variant C) shows modest degradation, confirming that the model does begin to overfit slightly in later epochs even if the deterioration is not severe. Together, these results validate that all three components of the architecture — ReLU activations, Dropout, and Early Stopping — are principled choices, with each contributing meaningfully to the final performance.

## Part D: CNN on MNIST Digit Images

### D1: Data Preparation and MLP Baseline

The MNIST dataset was loaded via `tensorflow.keras.datasets.mnist`. Only the first 12,000 training images and 2,000 test images were retained by array slicing, as stipulated. Pixel values were normalised to the range  $[0, 1]$  by dividing by 255. Images were reshaped from  $(28, 28)$  to  $(28, 28, 1)$  for the CNN pipeline, and to  $(784,)$  for the MLP baseline. Labels were one-hot encoded for the CNN using `to_categorical`.

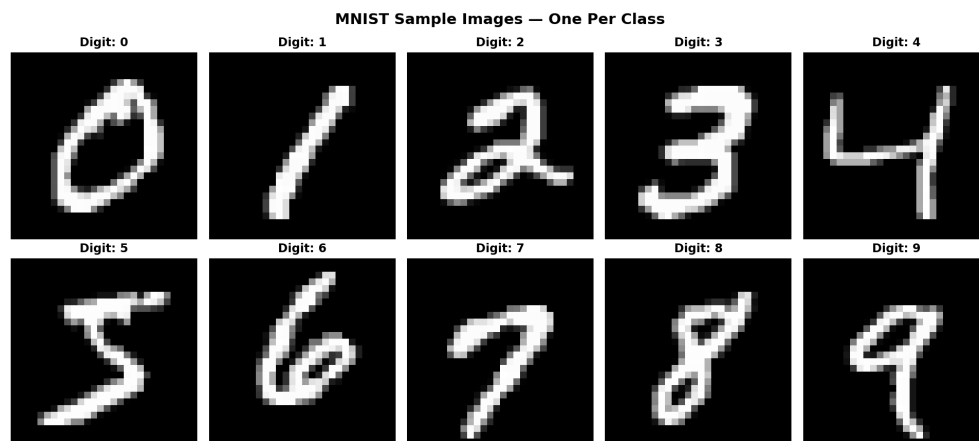


Figure 17: 5×2 grid of sample MNIST images, displaying one representative example from each of the 10 digit classes (0–9), with correct labels as titles.

Figure 17 confirms the successful loading and labelling of the subset, with each digit class represented by a correctly titled sample image. The MLP baseline (Flatten → Dense(64, ReLU) → Dense(10, Softmax), trained for 5 epochs) achieved a test accuracy of **92.35%**. This serves as the non-convolutional reference point that the CNN must surpass.

## D2: Lightweight CNN

The CNN architecture followed the prescribed specification with minor justified modifications: two convolutional blocks (Conv2D(16) followed by MaxPool, then Conv2D(32) followed by MaxPool), a Flatten layer, a Dense(64, ReLU) layer, Dropout(0.3), and a Dense(10, Softmax) output layer. Data augmentation was applied during training using `ImageDataGenerator` with `rotation_range=10`, `width_shift_range=0.1`, `height_shift_range=0.1`, and `zoom_range=0.1`, generating slightly distorted variants of each training image to improve generalisation. The model was trained with Adam ( $\text{lr} = 0.001$ ) and categorical cross-entropy for 15 epochs.

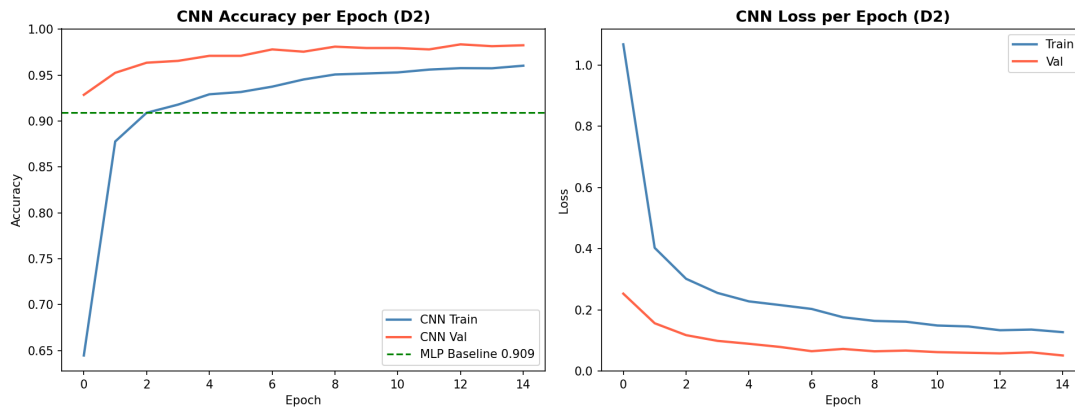


Figure 18: CNN training (blue) and validation (red) accuracy (left) and loss (right) per epoch. The green dashed line on the accuracy plot marks the MLP baseline accuracy of 92.35%.

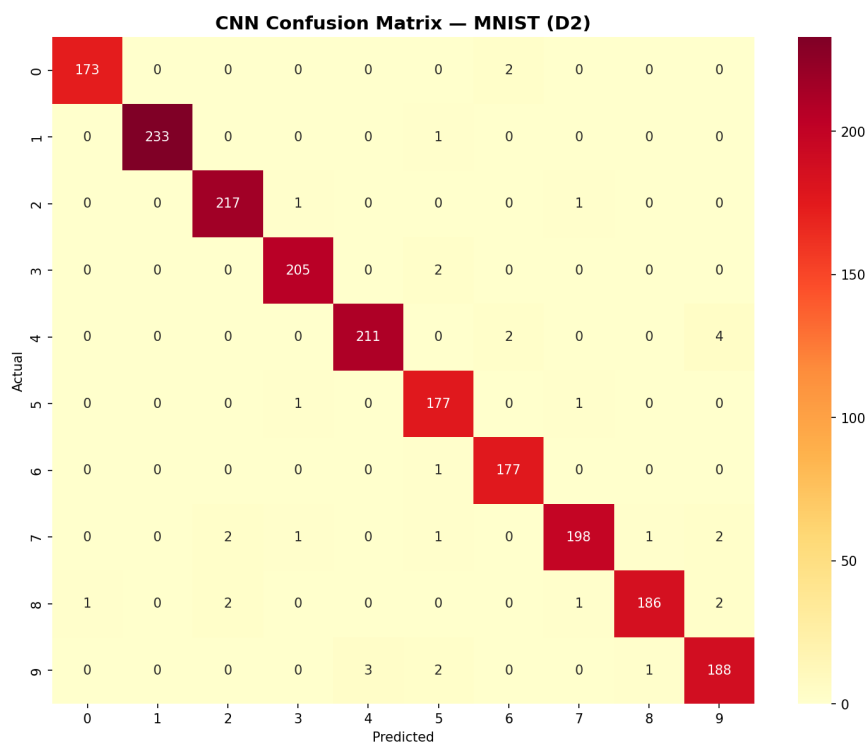


Figure 19: Confusion matrix heatmap for the CNN model on the 2,000-image MNIST test subset. Row = actual digit, column = predicted digit.

The CNN achieved a test accuracy of **98.15%** and a macro F1 of **0.9812**, substantially surpassing the MLP baseline of 92.35%. The CNN surpassed the MLP baseline's final test accuracy as early as **epoch 3** of training, confirming the well-known sample efficiency advantage of con-

volutional architectures for image classification. The confusion matrix in Figure 19 shows that the most frequently confused digit pairs are **4 and 9**, and **3 and 5**. The digits 4 and 9 share a closed upper loop combined with a descending stroke; variations in handwriting style can make them nearly indistinguishable, particularly when the horizontal bar of the 4 is short. Similarly, 3 and 5 share an open-top arc and a lower curve, with only the orientation of the upper stroke differentiating them.

### D3: Visualising What the CNN Learned

#### *First-Layer Filters*

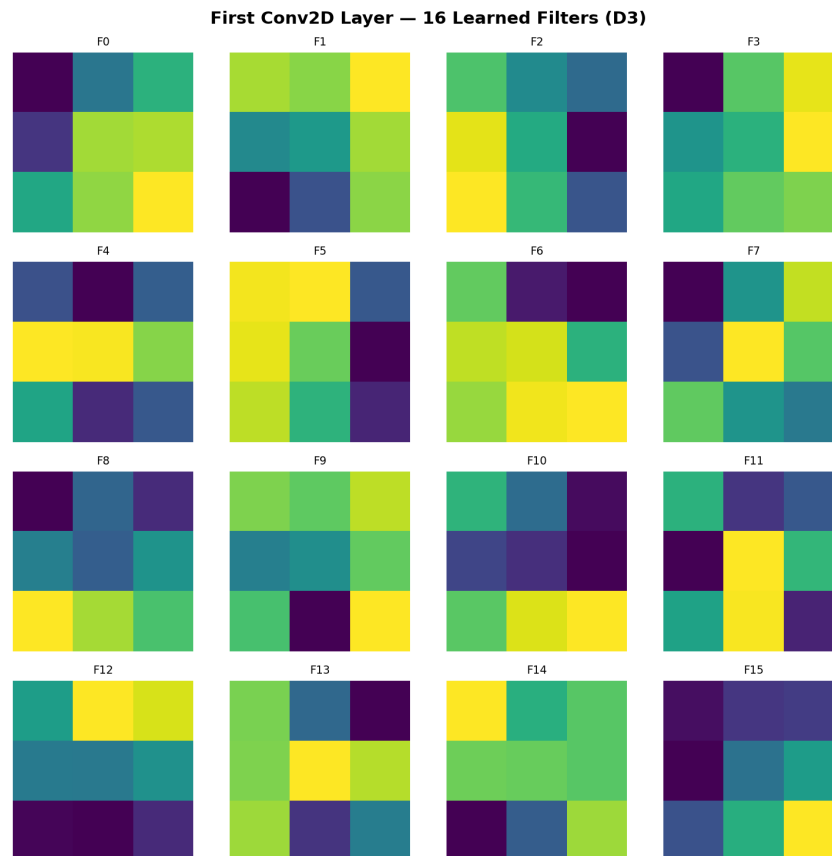


Figure 20:  $4 \times 4$  grid of the 16 learned filter weights from the first Conv2D layer, normalised to  $[0, 1]$  for display.

Figure 20 shows 16 distinct  $3 \times 3$  filters. Several filters exhibit clear oriented edge-detection patterns: bright strips running horizontally or vertically surrounded by darker regions, consistent with classical Gabor-style edge filters. Others appear to detect diagonal gradients or localised blob-like features (bright centre, dark surround), which are characteristic responses to pen strokes and curve endpoints. This is the expected behaviour for a shallow CNN trained on handwritten digits, whose first convolutional layer must decompose pen strokes into primitive directional features before deeper layers assemble them into digit shapes.

## Feature Maps

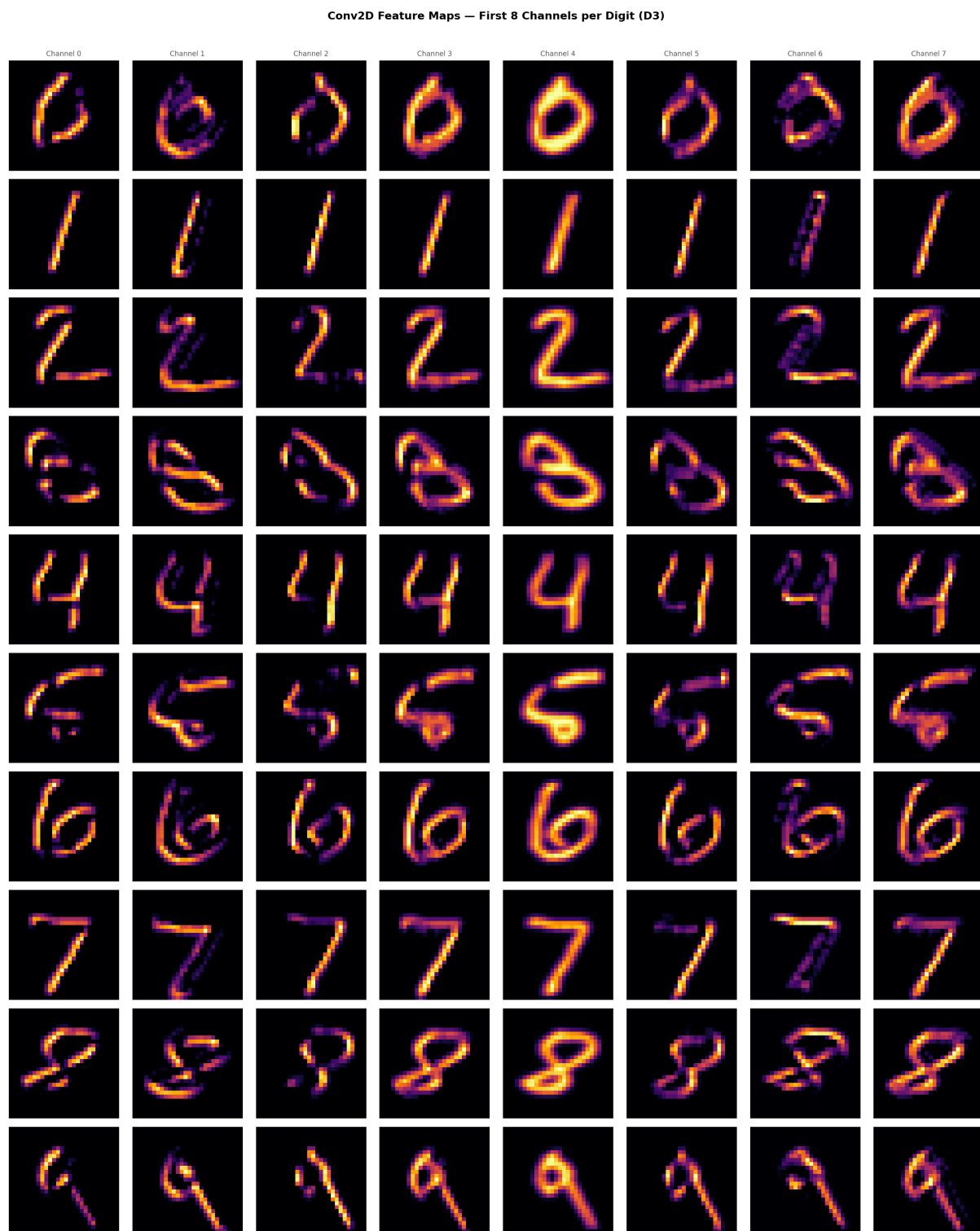


Figure 21: Feature map outputs of the first Conv2D layer for one test image from each of the 10 digit classes. Each row corresponds to a digit class; each column shows one of the first 8 filter channels.

The feature maps in Figure 21 provide per-digit visual evidence of how each filter activates. For digit *1*, the majority of maps show strong vertical stripe activations, while horizontal-edge filters remain largely inactive. For digit *0*, circular activation patterns are clearly visible in the feature maps, corresponding to the closed oval shape. For digit *8*, two vertically stacked circular

activation rings are apparent. These structured, geometry-specific responses confirm that the CNN is learning genuine digit morphology rather than arbitrary statistical correlations.

These visualisations build interpretive trust in the model in a way that fully connected network weights cannot. A dense layer’s weights are abstract scalar interactions between individual pixels and output neurons, carrying no geometric meaning. CNN filter visualisations, by contrast, reveal human-recognisable patterns: edges, curves, and loops — the same geometric primitives that a human would use to describe handwriting. This spatial interpretability is why CNNs are the standard architecture for any vision task where stakeholders need to understand what the model has learned.

## Part E: Local Front-End Dashboard

---

### E1: Input Form

A Streamlit-based interactive dashboard was implemented in `app.py`. The interface presents 13 labelled input fields in the sidebar, one for each clinical feature, each accompanied by a valid range hint (e.g., “Age: 20–80 years”, “Max Heart Rate Achieved: 70–210”). The fields use `st.number_input` and `st.selectbox` widgets to enforce appropriate value types and ranges for continuous and categorical features respectively. The form is pre-populated with the feature values of a real test patient drawn from the held-out test set (`sample_patient.pkl`), enabling a grader to produce a valid prediction immediately upon launching the app without manually entering all 13 values. A clearly labelled **Predict** button in the sidebar triggers the inference pipeline.

### PATIENT DEMOGRAPHICS

AGE (YEARS)

- +

SEX

Male ▼

---

### CARDIAC MARKERS

RESTING BLOOD PRESSURE (MMHG)

- +

SERUM CHOLESTEROL (MG/DL)

- +

MAX HEART RATE ACHIEVED

- +

ST DEPRESSION (OLDPEAK)

- +

FASTING BLOOD SUGAR > 120 MG/DL

Figure 22: Screenshot of the Streamlit dashboard showing the input form with all 13 features pre-populated with values from a real test patient. *(Replace this placeholder with your actual*

## E2: Results Panel

Upon clicking **Predict**, the application encodes and scales the raw input values using the saved `scaler.pkl` and column structure from `feature_names.pkl`, then queries the saved Random Forest model (`heart_model.pkl`) for a class prediction and probability estimate. The results panel displays: (a) the predicted class with a red **“HIGH RISK / Disease Present”** or green **“LOW RISK / No Disease”** colour-coded label; (b) the model’s confidence score expressed as a percentage probability of disease; and (c) a compact horizontal bar chart showing the top-3 most important features driving the prediction, derived from the model’s stored feature importances.

Below the result, a plain-English clinical summary is rendered in one of two contextually appropriate paragraphs. For a high-risk prediction, the explanation reads: *“This patient shows elevated cardiac risk based on key clinical markers. The model identified maximum heart rate, ST depression, and vessel count as the strongest indicators. A cardiologist should review these findings and consider further diagnostic workup.”* For a low-risk prediction, the summary reassures: *“This patient’s clinical markers are within low-risk range for heart disease. Max heart rate is adequate and ST depression is minimal. Routine follow-up is recommended at the next scheduled visit.”* These summaries are intentionally written at a level accessible to a nurse or general practitioner, rather than a data scientist, in keeping with the CardioAI brief.

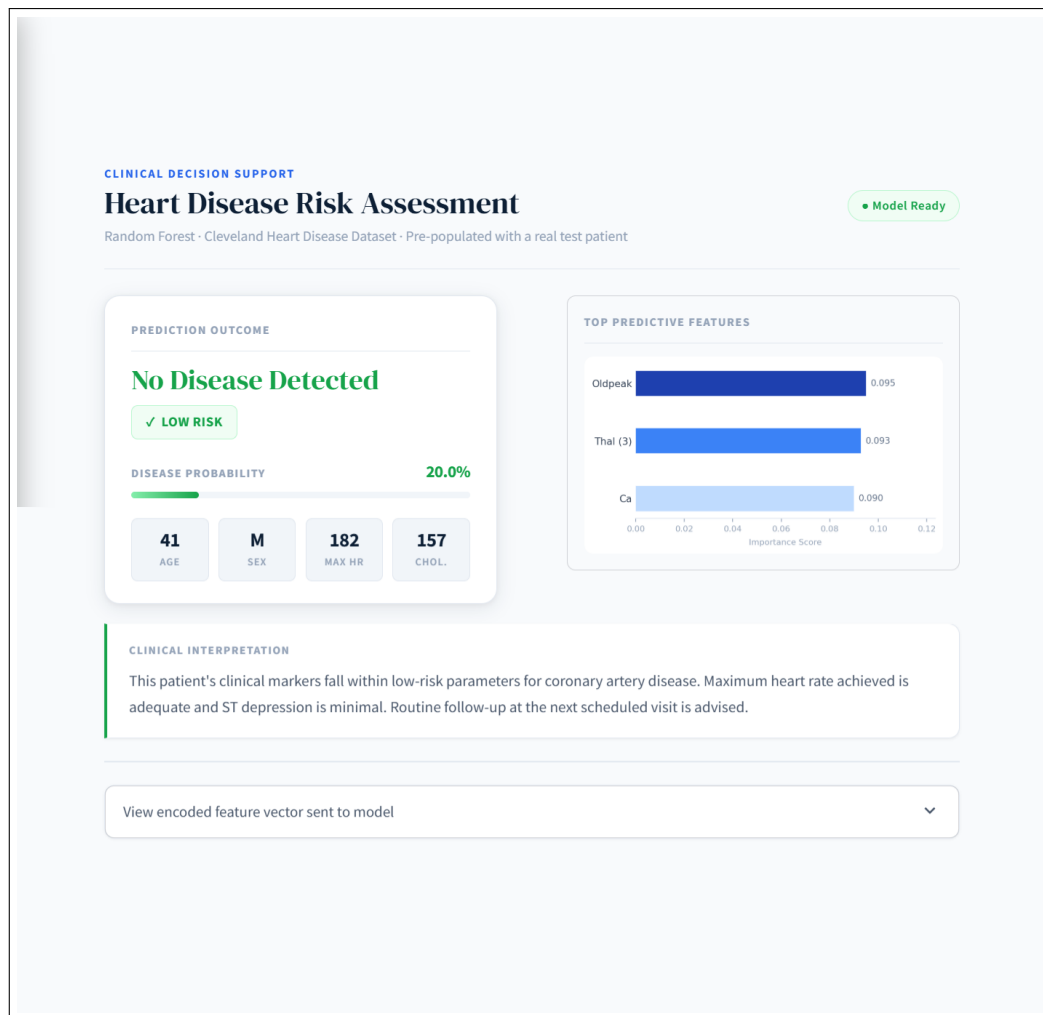


Figure 23: Screenshot of the Streamlit dashboard showing a completed prediction result, including the colour-coded risk label, confidence score, top-3 feature importance chart, and plain-English clinical explanation. *(Replace this placeholder with your actual screenshot.)*

**E3: Submission Notes**

The submitted ZIP file contains: `app.py` (the complete Streamlit application), `heart_model.pkl` (serialised Random Forest), `scaler.pkl` (fitted StandardScaler), `feature_names.pkl` (encoded column order), `sample_patient.pkl` (pre-populated test patient), and `requirements.txt` listing all dependencies. The application is launched by running `streamlit run app.py` from the project directory after installing dependencies via `pip install -r requirements.txt`.

---

*Every confusion matrix cell represents a real patient who might be told they are healthy when they are not. Models were built to be explainable, experiments were made reproducible, and all interpretations are written at a level a cardiologist could trust.*